3. Add a page that lists all `Venue` objects along with their corresponding rooms. You will need to iterate over the `room_set` query manager in the template.

Summary

- The Django ORM provides an object-oriented abstraction to a relational database.
- You inherit from `django.db.models.Model` to define a class to be mapped to the database.
- Your `Model` class uses fields to both define the type of data stored in the database and access to that data once queried.
- The `makemigrations` management command creates a Python file that describes the changes to the database needed by your models.
- The `migrate` management command runs any migration files that describe changes to models that have happened since the last invocation.
- Django's default database is SQLite, an embedded, single-file database.
- A database's query tool can be accessed using the `dbshell` management command.
- `Model` objects have a query manager named `.objects` that is used to query contents from the database for that model. The `.create()` method is used to create objects, while the `.first()`, `.last()`, and `.all()` methods return the first item, last item, and all the items for the object, respectively.
- A subset of values can be queried using the `.filter()` method, specifying key–value pairs for the filter. A value of `first_name="John"` returns all objects for which the `first_name` attribute is exactly `"John"`.
- The arguments to `.filter()` can be augmented to specify comparisons instead of exact matches. Suffixing a keyword with `__startswith` causes `.filter()` to return partial matches. A value of `first_name__startswith="J"` returns all objects for which the `first_name` field begins with the letter `"J"`.
- After modifying a `Model` instance, you call `.save()` on the object to update the database.
- The data for an object can be removed from the database using the `.delete()` method on the object itself. The Python proxy continues to exist after `.delete()` is called.
- Views can be parameterized to take arguments. Arguments can be given through URL query parameters or captured out of the URL itself.
- The `get_object_or_404()` shortcut retrieves an object from the database or raises an `Http404` exception in a single call.
- `Paginator` objects slice a query result into pages, allowing you to paginate the output in your templates.
- Django supports both one-to-many and many-to-many relations in the database. One-to-many relationships are defined using `ForeignKey` fields, while many-to-many relationships use the `ManyToManyField`.